

EventPlanner: Event Scheduling System

Introduction

- ✓ EventPlanner is a comprehensive platform designed to assist users in organizing events. It lets users sign up, create events (as organizers), invite attendees, and track attendance. The system supports quick searching and filtering so you can find relevant events and tasks by keywords, dates, or user role.
- ✓ Each team will be assigned a different technology stack (for the frontend, backend and database)
- ✓ The project features will be elaborated below.
- ✓ The project will be submitted as separate phases. Each phase deliverables are clarified below.

Event Planner Features (Functional Requirements)

1. User Management

- Users can create an account using their email address and password.
- Users can log in to their accounts using their credentials.

2. Event Management (Event Organizer Role only)

- Users can create new events with a title, date, time, location, and description.
- Users can view all events they have organized
- Users can view all events they are invited to.
- Users can invite others to attend or collaborate on an event.
- Users can delete events they have created.
- For each event of a user, the user must be marked as an “organizer” if he is the creator of the event or an “attendee”.

3. Response Management

- Attendees can indicate their attendance status for events (Going, Maybe, Not Going).
- Organizers can view the list of attendees and their statuses for each event.

4. Search and Filtering:

- Create an advanced search API to filter events and tasks based on keywords (event names, task descriptions), dates, and user roles.

Phases Description

Phase 0

The objective of this phase is to be familiar with the assigned technology stack. So only the “User Management” feature will be delivered (Sign up and Login).

Phase 0 Deliverables:

1. A text file **named as ID1_ID2_ID3_TA_NAME** containing team names and IDs, Links for the GitHub repositories for both the backend and frontend. Please ensure to add your TA as a collaborator to allow project download.
- ✓ **Any commits made after the project deadline will be treated as late submissions and will incur deductions.**

Phase 1

All the Project functional requirements will be delivered. Proper errors and exception handling should be considered.

Phase 1 Deliverables: One zip named as ID1_ID2_ID3_TA_NAME and the zip file contains:

1. A text file containing team names and IDs, Links for the GitHub repositories for both the backend and frontend. Please ensure to add your TA as a collaborator to allow project download.
 2. A Postman JSON file containing a collection for all backend APIs.
- ✓ **Any commits made after the project deadline will be treated as late submissions and will incur deductions.**

Phase 2

The primary goal of Phase 2 is to create three Docker images: one for the backend, one for the frontend, and one for the database. After building these images, you will need to create containers from them and manage the integration between the containers.

The project should operate as seamlessly as it did in Phase 1, but with one key difference: In Phase 1, the code was executed manually through the command line or an IDE. In Phase 2, the application will run through containers created from the Docker images, eliminating the need for manual execution. The project is expected to function correctly, just as it did in the earlier phase.

Note:

- ✓ Before beginning work on Phase 2, ensure you create a snapshot (copy) of your Phase 0 and Phase 1 code. Then, make a new copy of your repository to use for Phase 2 development.
- ✓ Use light base version to minimize the final image size.
- ✓ **All data must be persistent across runs (must use volumes).**
- ✓ Build the images before Phase 2 discussions and make sure that your containers are integrating properly.

Phase 2 Deliverables: One zip named as ID1_ID2_ID3_TA_NAME and the zip file contains:

1. A text file containing team names and IDs, Links for the GitHub repositories for both the backend and frontend.
2. A Postman JSON file containing a collection for all backend APIs.
3. Three docker files (frontend, backend and database)

Phase 3

The objective of this phase is to deploy your created app on openshift, to be available online.

Phase 3 Deliverables: One pdf file named as ID1_ID2_ID3_TA_NAME that contains

1. Team Names and IDs.
2. The app dynamic URL on OpenShift
3. Full screenshot of your infrastructure and connections between app layers on OpenShift.
4. Full screenshot of OpenShift website that shows the GitHub repo URL used in deployment.

Phases Deadline

Exact grading criteria will be announced before each phase.

Phase	Phase Weight	Deadline
Phase 0 (sign up and login)	10%	31 st of October
Phase 1 (full feature set)	20%	14 th of November
Phase 2 (docker files)	45%	5 th of December
Phase 3 (project deployment)	25%	27 th of December

Important Notes for All Teams

- ✓ Each phase has different deliverables. Please read the phases description well.
- ✓ **3- 4 students per team from the same lab or same TA.**
- ✓ Each team will be assigned a different technology stack.
- ✓ Please register your team to receive your designated technology stack at the **Team Registration Form** that will be announced on classroom.
- ✓ All team members must attend discussions at the designated time set by the TA. Any team member who misses the discussion will receive a grade of 0 out of 15 points.
- ✓ Each team member must have an equal share of features; workload distribution will be assessed.
- ✓ After registering, we will announce the assigned technology stack for all teams
- ✓ **Technology Stack Violation:** A penalty of 50% of the total points will be imposed for any violations.
- ✓ Projects must be submitted via Google Classroom.
- ✓ Deadline: check **Phases Deadline**
- ✓ **Late Submissions:** Not allowed.
- ✓ **Cheating Policy:** cheating cases get -2 grades for both teams.

Additional Resources: Suggested resources to help you get started.

- ✓ Go lang : <https://medium.com/@rasoky.maulana/create-a-crud-rest-api-using-mysql-golang-part-2-5af4942abdd>
- ✓ Python: <https://techarise.com/restful-api-in-python-mysql/>
- ✓ NodeJs: <https://www.bezkoder.com/node-js-rest-api-express-mysql/>
- ✓ React: <https://www.bezkoder.com/react-crud-web-api/>
- ✓ Angular <https://www.bezkoder.com/angular-16-crud-example/>
- ✓ Ruby on Rails: https://www.youtube.com/watch?app=desktop&v=oyjzi837wME&ab_channel=zayne